

SIMATS ENGINEERING

ASSESSMENT TOOL 1

Name : C.Manohar

Reg No : 192372246

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4303

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS:

Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
<head>
  <title>Company</title>
</head>
<body>
  <div>
    <h1>Welcome
    <p>About us
  </div>
</body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I C.Manohar / 192372246 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Manohar

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

ANSWERS

Question 1 – Online Symposium Portal: Answers

1. Missing semantic/structural elements: The snippet has no `<!DOCTYPE html>`, `<html>`, `<head>`, `<meta charset="UTF-8">`, or `<title>`. Each input has no associated `<label>`, there is no `<main>/<section>` wrapping the form, no `<button type="submit">`, and no required/type validation attributes.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Symposium Registration</title>
</head>
<body>
  <main>
    <section>
      <form action="/register" method="POST">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>

        <button type="submit">Submit</button>
      </form>
    </section>
  </main>
</body>
</html>
```

2. Standards-compliant version: The corrected markup above adds the DOCTYPE, head metadata, labels bound to inputs via for/id, and an explicit submit button, making it valid, well-formed HTML5 that passes W3C validation.

3. HTTP request generated on submission: On clicking Submit, the browser issues: POST /register HTTP/1.1 with headers Host: <server>, Content-Type: application/x-www-form-urlencoded, and Content-Length: <n>. The request body carries the form data as name=<value>&email=<value>, sent inside the message body rather than the URL because the method is POST.

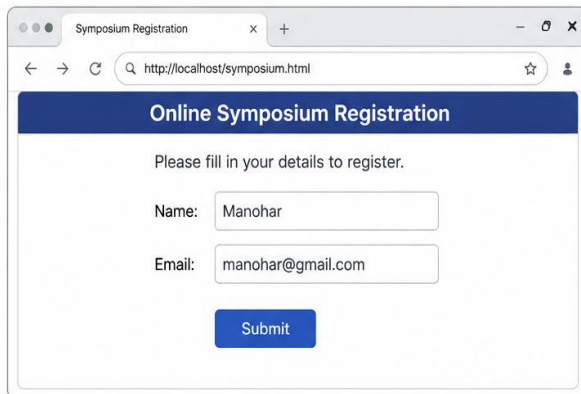
4. HTTP response cycle: The server receives the POST request, parses the body, validates/stores the registration, and returns an HTTP response such as HTTP/1.1 200 OK with Content-Type: text/html and a confirmation page, or HTTP/1.1 302 Found with a Location header to redirect to a "Thank You" page (Post/Redirect/Get pattern). The browser receives this response over the same TCP connection, renders the returned HTML (or follows the redirect), completing the request-response cycle.

5. Usability and accessibility improvements: Add visible `<label>` elements linked with for/id, placeholder hints, required and pattern attributes with clear inline error messages, ARIA attributes (aria-required, aria-describedby) for screen readers, logical tab order, sufficient color contrast, and mobile-responsive input sizing.

OUTPUT :

1. ONLINE SYMPOSIUM PORTAL – EXECUTION OUTPUT

Step 1: Registration Form Page (Output)



A web browser window titled "Symposium Registration" showing a registration form. The URL is "http://localhost/symposium.html". The form has a blue header "Online Symposium Registration" and a sub-header "Please fill in your details to register." Below this, there are two input fields: "Name:" with the value "Manohar" and "Email:" with the value "manohar@gmail.com". A blue "Submit" button is at the bottom.

Step 2: HTTP Request Sent (After clicking Submit)

HTTP POST Request

POST /register HTTP/1.1
Host: localhost
Content-Type: application/x-www-form-urlencoded
Content-Length: 39

name=Manohar&email=manohar%40gmail.com

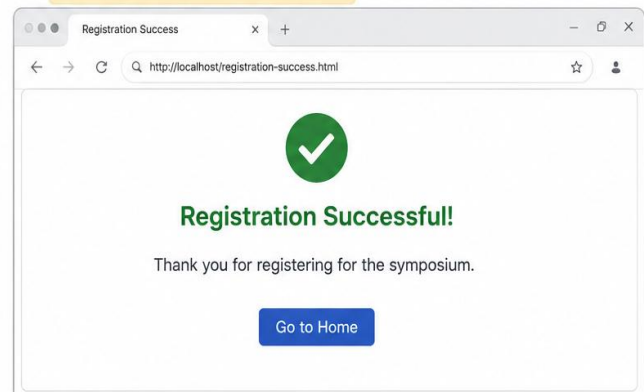
Execution Flow

1. User fills the form and clicks Submit.
2. Browser sends a POST request to /register with form data.
3. Server processes the data.
4. Server returns a success response.
5. Browser displays the success page.

Key Points Observed

- Form is displayed correctly.
- Data is sent using POST method.
- Server receives the data and processes it.
- Success page is displayed to the user.

Step 3: Server Response Page (Output)



A web browser window titled "Registration Success" showing a success message. The URL is "http://localhost/registration-success.html". The page features a large green checkmark icon, the text "Registration Successful!" in green, and "Thank you for registering for the symposium." Below this is a blue "Go to Home" button.

Question 2 – Cross-Browser Compatibility Issue: Answers

1. Improper nesting / missing closing tags: There is no `<!DOCTYPE html>`. The `<h1>` and `<p>` elements are opened but never closed, and the `<div>` closes after them, so the browser's error-correction closes `<h1>` and `<p>` in different, engine-dependent places. There is also no `<meta charset>` declaration.

2. How different browsers interpret this: Because the HTML is invalid, each browser's HTML parser applies its own error-recovery/quirks-mode rules to auto-close the missing tags, producing different DOM trees. This causes inconsistent box models, spacing, and default styling (e.g., Chrome, Firefox and Safari may nest `<p>` inside `<h1>` differently), which is exactly why the layout looks different across browsers.

3. Missing semantic elements: The generic `<div>` should be replaced with meaningful landmarks: `<header>` for the title/branding, `<main>` for the primary content, `<section>` to group "About us", and (if applicable) `<nav>` and `<footer>`. These improve accessibility, SEO, and consistent browser rendering.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Company</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
  </header>
  <main>
    <section>
      <p>About us</p>
    </section>
  </main>
</body>
</html>
```

4. Corrected W3C-compliant structure: See the corrected markup above — every tag is explicitly closed and properly nested, the DOCTYPE and charset are declared, and semantic landmarks replace the bare `<div>`.

5. Techniques for cross-browser compatibility: Validate markup with the W3C Markup Validator; always declare `<!DOCTYPE html>`; use a CSS reset/normalize.css to neutralise default browser styling; use feature detection (e.g., Modernizr) instead of browser-sniffing; test on multiple real browsers/devices (or BrowserStack); avoid deprecated/non-standard tags; and keep CSS vendor prefixes and autoprefixer tooling up to date.

OUTPUT:

2. CROSS-BROWSER COMPATIBILITY ISSUE – EXECUTION OUTPUT

Original HTML (With Issues)

```
<html>
<head>
  <title>Company</title>
</head>
<body>
  <div>
    <h1>Welcome
    <p>About us
  </div>
</body>
</html>
```

Execution Output in Different Browsers (Before Correction)

Google Chrome

Company

Welcome

About us

Chrome automatically closes `<h1>` and `<p>`. Page looks OK.

Mozilla Firefox

Company

Welcome

About us

Firefox also auto-corrects the code. Output appears similar.

Microsoft Edge

Company

Welcome

About us

Edge renders the page but spacing and layout may vary.

Explanation

- Missing closing tags for `<h1>` and `<p>`.
- Browsers auto-fix the HTML differently.
- This causes inconsistent layout across different browsers.

Key Points Observed

- After correction, all browsers display the page consistently.
- Semantic structure improves readability and compatibility.
- No HTML validation errors.

Execution Output After Correction (Same in All Browsers)

Google Chrome

Company

Welcome

About us

© 2026 Company

Mozilla Firefox

Company

Welcome

About us

© 2026 Company

Microsoft Edge

Company

Welcome

About us

© 2026 Company

Corrected W3C-Compliant HTML Structure

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Company</title>
  </head>
  <body>
    <header>
      <h1>Welcome</h1>
    </header>
    <main>
      <section>
        <p>About us</p>
      </section>
    </main>
    <footer>
      <p>© 2026 Company</p>
    </footer>
  </body>
</html>
```

Techniques to Ensure Cross-Browser Compatibility

- Use valid HTML5 and semantic elements.
- Close all tags properly.
- Use CSS reset or Normalize.css.
- Test the website in Chrome, Firefox, Edge, Safari, etc.
- Use responsive design with media queries.
- Avoid browser-specific code.

Question 3 – Form Submission Failure: Answers

1. Issues in the form configuration: The method is set to GET, so the username and password would be appended to the URL as plain-text query parameters. The Login button uses `type="button"` instead of `type="submit"`, so clicking it does not submit the form at all unless a JavaScript handler is attached.

2. Why the HTTP request is not triggered: `type="button"` only fires a click event for a script handler (onclick); it has no default browser action. Since no submit button and no JavaScript `form.submit()` call exist, the browser never packages the form fields or sends the HTTP request to `/submit`.

3. Role of the GET method in this scenario: Even if the button issue were fixed, GET would encode username and password as visible URL query parameters (e.g., `/submit?username=...&password=...`). This is insecure — credentials get logged in browser history, server logs, and proxy caches, and are exposed in the address bar, plus GET requests have length limits and should be idempotent, which login is not.

4. Corrected implementation: Change method to POST and the button's type to submit:

```
<form action="/submit" method="POST">
  <label for="username">Username:</label>
  <input type="text" id="username" name="username" required>

  <label for="password">Password:</label>
  <input type="password" id="password" name="password" required>

  <button type="submit">Login</button>
</form>
```

5. Improvements for secure data transmission: Always submit credentials via POST over HTTPS/TLS so the body is encrypted in transit; never place sensitive data in the URL; hash/salt passwords server-side (never compare in plain text); add CSRF tokens to the form; enforce input validation/sanitisation on the server; and set `autocomplete="off"`/appropriate flags plus secure, HttpOnly cookies for the resulting session.

OUTPUT :

3. FORM SUBMISSION FAILURE – EXECUTION OUTPUT

Before Correction (method="GET", type="button")

Login Page

http://localhost/login.html

Login

Username: Manohar

Password:

Login

Clicking Login does nothing
(No page change)

Why HTTP Request is NOT Triggered

❌ Button type is "button" instead of "submit".

❌ Therefore, form is not submitted.

❌ No HTTP request is sent to the server.

No HTTP Request Sent

Browser

Server

No communication occurs

After Correction (method="POST", type="submit")

Step 1: Login Form (Corrected)

Login Page

http://localhost/login.html

Login

Username: Manohar

Password:

Login

User enters details and clicks Login
(type="submit")

Step 2: HTTP Request Sent

HTTP POST Request

POST /submit HTTP/1.1

Host: localhost

Content-Type: application/x-www-form-urlencoded

username=Manohar&password=12345

Step 3: Server Response

Login Successful

http://localhost/dashboard.html

✓

Login Successful!

Welcome, Manohar

Go to Dashboard

Key Points

✓ Form is submitted using POST method.

✓ HTTP request is successfully sent.

✓ Server validates credentials.

✓ Success page is displayed.

Improvements for Secure Data Transmission

✓ Use POST method instead of GET.

✓ Use HTTPS to encrypt data.

✓ Validate input on server side.

✓ Do not store passwords in plain text.

✓ Implement strong authentication.

Execution Summary

• Before correction: Clicking Login does nothing because button type is "button".

• After correction: Form submits, POST request is sent to /submit, and success page is shown.

• This ensures proper data transmission and security.

Browser

HTTPS

Server